Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Владимирский государственный университет
имени Александра Григорьевича и Николая Григорьевича Столетовых»
(ВлГУ)
Кафедра ФиПМ

ЛАБОРАТОРНАЯ РАБОТА №2
по дисциплине «Операционные системы»
на тему:
«РАБОТА С КОМАНДНОЙ СТРОКОЙ В LINUX.
АДМИНИСТРИРОВАНИЕ.»

Выполнил:
ст. группы ИТ-124
Горностаев А.А.
Принял:
ассистент каф. ФиПМ
Болачков А. В.

Владимир 2026

Цель работы.

Знакомство с командной строкой Linux, изучение основных команд. Получить навыки администрирования пользователей.

Постановка задачи.

Часть 1.

1. Определить путевое имя рабочего каталога. Как обозначается корневой каталог? Какое путевое имя получили (относительное или абсолютное)?

2. Создать в начальном каталоге два подкаталога. Просмотреть содержимое рабочего каталога. Просмотреть содержимое родительского каталога, не переходя в него.

3. Перейти в системный каталог. Просмотреть его содержимое. Просмотреть содержимое начального каталога. Вернуться в начальный каталог.

4. Удалить созданные ранее подкаталоги.

Получить информацию по командам ls и cd с помощью утилиты man. Изучить структуру man-документа.

5.Получить краткую информацию по командам ls и cd с помощью команды whatis и apropos. В чем различие?

6.То же, что и в п.5, только с помощью команды info.

Часть 2

1.Выполнить команды top, free, ps с различными
опциями.

2. Войти в свой домашний каталог. Для этого нужно сделать команду cd ~

Вы находитесь в своем рабочем каталоге. Здесь хранятся ваши пользовательские файлы и настройки программ, которые вы используете.

3. Создать следующую структуру каталогов и файлов

1) в домашнем каталоге создать каталог inform

2) Перейти в каталог и inform создать в нем каталог lab1

3) Внутри каталога lab1 создать каталог catalog1, файл file1(например, используя команду echo), каталог catalog2. Перейти в каталог catalog2.

4) Внутри каталога catalog2 создать файлы file3 и file4 , каталог catalog3

5) Внутри каталога catalog3 создать файл file5, жесткую ссылку на файл file1, жесткую ссылку на каталог catalog2.

6) Создать в каталоге lab1 символичесткую ссылку s_link на файл file5

Часть 3.

1. Создайте в домашнем каталоге следующую структуру подкаталогов (существующие каталоги не удаляйте!)

a. домашний каталог ----|

1. |-ВашаФамилия-|

a. |-1-|

b. | |-2

c. | |-3

d. |

e. |-4

2. Скопируйте файл /etc/group в каталог 1 используя абсолютные имена копируемого файла и каталога назначения.

3. Скопируйте файл /etc/group в каталог 2 используя абсолютное имя копируемого файла и относительное имя каталога назначения.

4. Скопируйте файл /etc/group в каталог 3 используя относительные имена копируемого файла и каталога назначения.

5. Скопируйте файл /etc/group в каталог 4
используя абсолютные имена
копируемого файла и относительное имя каталога назначения с использованием специального символа ~.

6. При помощи одной команды зайдите в каталог 3.

7. Удалите файл group из каталога 4 при помощи одной команды.

8. Перейдите в свой домашний каталог. Удалите каталоги 1 и 4.

9. Выведите первые и последние 13 строк файла /etc/group.

Часть 4

1. Создать группы:

- workers,
- teachers,
- students

2. Создать пользователей kit_[номер варианта]_ N, где N =1, 2, .., 5, uid учетной записи должен быть равен 1000+N.

Пользователей с N равным 1 и 2 добавить в группу workers.

Пользователей с N равным 3, 4 и 5 добавить в группу students

3. Создать пользователя teacher_[номер варианта]. В комментарии к учетной записи должны быть имя и фамилия студента. uid учетной записи должен быть равен 3000. Пользователя добавить в группу teachers.

4. Создать директорию labs в корневом каталоге. В нем создать каталоги library и tests

5. Создать файлы book_[фамилия студента]_N и поместить их в library

6. Создать текстовый файл test_[имя студента], и поместить в tests. Файлы должны содержать скрипт на создание пользователя user[номер варианта] и задание ему пароля pass[номер варианта]. Сделайте эти файлы исполняемыми для пользователей группы students.

7. В директории labs создать файл list, который должен содержать список файлов директории /etc.

8. Дать право на изменение файла только пользователю teacher_[номер варианта], а на чтение пользователям группы workers.

9. Настроить права доступа к каталогу library и tests, таким образом, чтобы пользователи группы teachers могли изменять и создавать там файлы, а пользователи группы students имели доступ на чтение.

Теоретическая часть

Для его изучения включите терминал (Приложения > Стандартные > Терминал).

Работу ОС LINUX можно представить в виде функционирования множества взаимосвязанных процессов. При загрузке системы сначала запускается ядро (процесс 0), которое в свою очередь запускает командный интерпретатор shell (процесс 1).

Взаимодействие пользователя с системой LINUX происходит в интерактивном режиме посредством командного языка. Оболочка операционной системы – shell – интерпретирует вводимые команды, запускает соответствующие программы (процессы), формирует и выводит ответные сообщения.

Shell - это интерфейс, обеспечивающий взаимодействие между ядром и пользователем.

Интерфейс shell очень прост. Обычно он состоит из приглашения, по которому пользователь вводит команды и нажимает клавишу Enter. Строка, в которой вы набираете команду, называется командной строкой. Shell не только интерпретирует команды, но и создает среду, которую вы можете конфигурировать и программировать. У shell есть свой язык программирования, который позволяет писать программы, содержащие достаточно сложные последовательности команд Linux. Язык программирования shell обладает многими свойствами обычного языка программирования, в частности в нем предусмотрено использование циклов и условных переходов. Каждому пользователю системы Linux предоставляется свой собственный пользовательский интерфейс, или shell. Пользователи могут модифицировать свои shell в соответствии с конкретными потребностями. В этом смысле shell пользователя функционирует скорее как операционная среда, которой пользователь может управлять по своему усмотрению. За последние годы разработано несколько разновидностей shell. Сейчас используются в основном три варианта: Bourne,

Korn и C-shell. Bourne-shell был разработан в Bell Labs для System V. C-shell разработан для версии BSD. Kom-shell - это усовершенствованный вариант Bourne-shell. В современных версиях Unix, включая Linux, представлены все три вышеназванных shell, что дает пользователю возможность выбора. В Linux, однако, используются расширенные или общедоступные версии этих shell: Bourne Again, TC-shell и Public Domain Korn. При запуске ОС Linux активизируется Bourne Again Shell, модифицированная версия Bourne. Отсюда можно переключаться в другие shell.

Файловая структура: каталоги и файлы

В операционной системе Linux все файлы организованы в каталоги, которые, в свою очередь, иерархически соединены друг с другом, образуя одну общую файловую структуру.

При обращении к файлу необходимо указывать не только его имя, но и место, которое он занимает в этой файловой структуре. Можно создавать любое количество новых каталогов, добавляя их к файловой структуре. Команды работы с файлами ОС Linux могут выполнять сложные операции, например, перемещение и копирование целых каталогов вместе с их подкаталогами. Такие команды, как find, cp, mv и ln, позволяют находить файлы, копировать их и перемещать из одного каталога в другой, а также создавать ссылки.

Файлы в операционной системе Linux организованы в иерархическую систему каталогов. Каталог может содержать файлы и другие каталоги. Из за сходства с деревом структуру каталогов часто называют древовидной структурой. Если быть более точным, то эта структура скорее похожа не на дерево, а на перевернутый вверх корнями куст. Ствола здесь нет, и изображается дерево перевернутым, при этом корень находится наверху. Вниз от корня отходят ветви. Каждая ветвь отходит только от одной ветви, а от нее самой может отходить множество ветвей нижнего уровня. В этом смысле данную структуру можно назвать структурой "родители-потомки".

Аналогичным образом любой каталог является подкаталогом другого каталога. Каждый каталог может содержать множество подкаталогов, но сам должен быть потомком только одного родительского каталога. Вверху файловой системы находится корневой каталог (обозначается символом "косая черта"), от которого ответвляются другие каталоги. Каждый каталог может содержать несколько других каталогов или файлов, но родительский каталог у него всегда бывает только один.

Файловая структура ОС Linux разветвляется на несколько каталогов, начиная с корневого, /.

В корневом каталоге имеется несколько системных каталогов, которые содержат файлы и программы, относящиеся к самой ОС Linux. Корневой каталог, кроме того, содержит каталог home, который может содержать начальные каталоги всех пользователей системы. Начальный каталог каждого пользователя, в свою очередь, будет включать в себя каталоги, который пользователь создает для своих нужд. Каждый из этих каталогов тоже может содержать каталоги. Все эти вложенные каталоги ответвляются от начального каталога пользователя.

Получить доступ к каталогу можно либо по имени, либо сделав его каталогом по умолчанию.

Каждому каталогу при создании присваивается имя. Этим именем можно пользоваться для доступа к файлам, находящимся в данном каталоге. Если при проведении какой-либо операции над файлами имена каталогов не указываются, то используется каталог по умолчанию, который называют рабочим каталогом. В этом смысле рабочий каталог – это каталог, в котором вы в данный момент работаете. При регистрации в системе в качестве рабочего принимается ваш начальный каталог, имя которого обычно совпадает с вашим регистрационным именем.

Путевые имена

Имя, которое дается каталогу или файлу при его создании, не является полным.

Полным именем каталога является его путевое имя. Иерархические связи, существующие между каталогами, образуют пути, и эти пути можно использовать для и однозначного указания каталога или файла и обращения к нему.

Можно сказать, что каждый каталог в файловой структуре имеет собственный уникальный путь. Фактическое имя, которым система обозначает каталог, всегда начинается с корневого каталога и состоит из имен всех каталогов, ведущих к данному каталогу.

В ОС Linux путевое имя каталога состоит из имен всех каталогов, образующих путь. Эти имена отделяются друг от друга символами "косая черта". Косая черта перед первым каталогом пути обозначает корневой каталог(/).

Путевые имена могут быть абсолютными и относительными.

Абсолютное путевое имя - это полное имя файла или каталога, начинающееся символом корневого каталога.

Относительное путевое имя начинается символом рабочего каталога и представляет собой обозначение пути к файлу относительно вашего рабочего каталога.

Системные каталоги

Корневой каталог, являющийся началом файловой структуры ОС Linux, содержит ряд системных каталогов. Системные каталоги содержат файлы и программы, служащие для управления системой и ее сопровождения. Многие из этих каталогов содержат подкаталоги с программами, предназначенными для выполнения конкретных задач.

/bin

bin - это сокращенно от `binaries' (т.е. двоичные или выполняемые файлы). Здесь находится много важных системных программ. Большинство основных команд Unix находятся в этом каталоге.

/dev

"Файлы" в dev известны как драйверы устройств - они используются для доступа к устройствам и ресурсам системы, таким как диски, модемы, память и т.д. Например, вы можете читать данные из файла, точно также вы можете читать входные сигналы от мыши, имея доступ к /dev/mouse. Имена файлов, начинающиеся на fd - это дисководы гибких дисков. fd0 - первый дисковод, fd1 - второй. Различные /dev/ttys, /dev/cua устройства используются для доступа к последовательным портам. Например, /dev/ttys0 относится к `COM1' под MS-DOS.

Устройства /dev/cua относятся к "звонящим" (``callout") устройствам, которые

используются совместно с модемами.

Устройства, имена которых начинаются с hd, имеют доступ к жестким

дискам. /dev/hda относится ко всему первому жесткому диску, а hda1 только к первому разделу /dev/hda.

Устройства с именами /dev/tty относятся к "виртуальным консолям" вашей системы (доступ путем нажатия alt-F1, alt-F2 и т.д.). /dev/tty1 соответствует первой, /dev/tty2 соответствует второй и т.д.

Устройства, чьи имена начинаются на /dev/pty, это "псевдотерминалы". Они используются для входа с удаленных "терминалов". Например, если ваша машина в сети, вход к вам по telnet будет использовать одно из устройств /dev/pty.

/etc

etc содержит файлы конфигурации системы. Например /etc/passwd(файл паролей), /etc/groups(файл групп), /etc/rc (командный файл инициализации) и т.д.

/sbin

В sbin находятся важные исполняемые системные файлы, используемые системным администратором.

/home

home содержит домашние каталоги пользователей.

/lib

lib содержит образы разделяемых библиотек (shared library images). Эти файлы

содержат код, который могут использовать многие программы. Вместо того, чтобы каждая программа имела свою собственную копию этих выполняемых файлов, они хранятся в одном общедоступном месте – в /lib. Это позволяет сделать выполняемые файлы меньше и сэкономит место в системе.

/proc

proc - это "виртуальная файловая система" procfs, в которой файлы хранятся в памяти, а не на диске. Они связаны с различными процессами, происходящими в системе, и позволяют получить информацию о том, что делают программы и процессы в указанное время. /tmp

Многие программы нуждаются в создании рабочих файлов, которые нужны короткое время. Каноническое место для этих файлов в /tmp (там обычно чаще проводится уборка мусора).

/usr

usr - состоит из ряда подкаталогов, которые в свою очередь содержат наиболее важные и полезные программы и файлы конфигурации, используемые системой.

Различные каталоги, описанные выше, необходимы для нормального функционирования системы, но большинство вещей, содержащихся в /usr необязательны для системы.

/usr/bin - для различных программ UNIX. Он содержит большинство выполняемых программ, которых нет ни в каких других местах, например, в том же /bin их нет.

/usr/etc – также как и /etc, содержит всевозможные системные программы и конфигурационные файлы.

/usr/include - содержит include-файлы(header - файлы) для компилятора Си.

/usr/lib - содержит библиотеки -"заглушки" и "статические" библиотеки,

эквивалентные файлам из /lib. При компиляции программа "связывается" с библиотеками, находящимися в /usr/lib, которые в свою очередь направляют программы обращаться в /lib, если им нужен актуальный код. Кроме того, многие другие программы хранят в /usr/lib свои конфигурационные файлы.

/usr/local - в большой степени похож на /usr - он содержит различные программы и файлы, несущественные для системы /usr/man - содержит страницы Руководства. Здесь два подкаталога для каждого "раздела" Руководства. (С помощью команды "man man" вы можете получить более подробную информацию). Например, /usr/man/man1 содержит исходные тексты (неотформатированный оригинал) страниц Руководства в разделе 1 и usr/man/cat1 содержит отформатированные страницы для раздела 1. /usr/src - содержит исходные коды (неоткомпилированные программы) для различных программ вашей системы. Наиболее важная вещь здесь это каталог /usr/src/linux, в котором содержатся исходные коды ядра Linux.

/var

var содержит каталоги, которые часто меняются в размере или имеют тенденцию быстро расти. К числу таких каталогов относятся:

/var/adm - содержит различные файлы, интересные системному администратору, специфические системные файлы, фиксирующие ошибки и проблемы, возникающие в системе. Другие файлы фиксируют входы в систему, как и неудачные попытки войти.

/var/spool - содержит файлы, которые предварительно формируются для других программ. Например, если ваша машина подключена к сети, входная почта будет помещаться в /var/spool/mail до тех пор, пока вы не прочитаете ее или не удалите.

Входящие и исходящие новости помещаются в /var/spool/news и т.д.

Зарегистрировавшись системе, пользователь попадает в свой начальный каталог. Имя, присвоенное этому каталогу системой, совпадает с регистрационным именем пользователя.

Все файлы, создаваемые для нового пользователя, помещаются в начальный каталог. В этом каталоге можно создавать подкаталоги и размещать в них файлы.

Основные команды терминала

Запустить Терминал (Приложения > Стандартные > Терминал). Эта программа предназначена для выполнения функций командной строки ОС Linux. Здесь в интерактивном режиме вы можете выполнять любые команды и программы, зарегистрированные в системе.

Кратко перечислим основные команды (программы) ОС Linux.

1. Вывод справки по команде (man)

man <имя изучаемой команды>

man (от англ. manual — руководство) — команда Unix, предназначенная для форматирования и вывода справочных страниц.

Ниже приведен пример, в котором пользователь вызывает из диалогового руководства информацию о команде ls ,

$ man ls

2. Список работающих процессов (top)

top

консольная команда UNIX-совместимых операционных систем, список работающих в данный момент процессов и информацию о них. Команда top показывает список работающих в данный момент процессов и информацию о них, включая использование ими памяти и процессора. Список интерактивно формируется в реальном времени. Чтобы выйти из программы top, нажмите клавишу [q].

3. Количество памяти (free)

free [-b | -k | -m] [-o] [-s delay] [-t] [-V]

Показывает общее количество свободной и используемой физической памяти и памяти отведенной для свопирования в системе, так же и совместно используемую память и буфера используемые ядром.

Опции :

-b показывает количество памяти в байтах; опция -k (по умолчанию) поазывает
количество пвмяти в килобайтах; Опция -m показывает количество памяти в мегабайтах.
-t показывает строки содержащие полное количество памяти.
-o запрещает показывать строки относящиеся к "массиву буфера" . Если не определено
отнять/добавить память буферов из/в используемую/свободнуб память (соответственно!).
-s разрешает безостановочно выводить информацию с промежутком в delay секунд.
-V показывает информацию о версии программы.
4. Отчет о работающих процессах (ps)
ps [опции]
Команда ps выводит в стандартный вывод информацию о текущем состоянии процессов.
Опции
-a все терминальные процессы
-e все процессы.
-g список выбирать процессы по списку лидеров групп.
-p список выбирать процессы по списку идентификаторов процессов.
-t список выбирать процессы по списку терминалов.
-u список выбирать процессы по списку идентификаторов пользователей.
f генерировать полный листинг.
-l генерировать листинг в длинном формате.
5. Выдача информации о файлах или каталогах (ls)
Синтаксис команды:
ls [флаги] [имя ...]
Команда ls для каждого имени каталога распечатывает список входящих в этот каталог файлов; для файлов - повторяется имя файла и выводится

дополнительная информация в соответствии с указанными флагами. По умолчанию имена файлов выводятся в алфавитном порядке. Если имена не заданы, выдается содержимое текущего каталога. Если заданы несколько аргументов, то они сортируются по алфавиту, однако сначала всегда идут файлы, а потом каталоги с их содержимым.

Команда ls -l позволяет получить подробную информацию о файле. Сначала указываются права доступа, затем количество ссылок, имя владельца файла, имя группы, к которой он относится, размер файла в байтах, дата и время последнего изменения и, наконец, имя файла. Имя группы обозначает группу, которой предоставляется доступ по категории "группа". Например (см. ниже), тип файла mydata - обычный файл. У него всего одна ссылка; это говорит о том, что у файла нет других имен. Имя владельца - chris, оно совпадает с регистрационным именем данного пользователя. Имя группы - weather. Вероятно, есть и другие пользователи, входящие в эту группу. Размер файла - 207 байт. Последний раз его корректировали 20 февраля в 11:55. Имя файла — mydata.

```
   Права доступа                                   Дата и время
       |                                            Последнего
       |                                            изменения
 Тип   | Количество    Имя        Имя    Размер       |        Имя
файла  | ссылок      владельца   группы  файла         |       Файла
 |     |      |          |           |    в байтах      |         |
 |  -----    |          |           |       |    ----------     |Ц
 v |     |   v          v           v       v   |          |    v
 -rw-r--r-- 1       chris       weather   207 Feb  20  11:55 mydata
Если вы хотите получить подобную информацию обо всех файлах,  на-
ходящихся в данном каталоге, дайте команду ls -l без аргумента:
$ ls -l
-rw-r--r-- chris weather 207 Feb 20 11:55 mydata
-rw-rw-r-- chris weather 568 Feb 14 10:30 today
-rw-rw-r-- chris weather 308 Feb 17 12:40 monday
```

Практическая часть

Часть 1.

1. Определить путевое имя рабочего каталога. Как обозначается корневой каталог? Какое путевое имя получили (относительное или абсолютное)?

```
alexander@alexander-PC: ~
alexander@alexander-PC:~$ pwd
/home/alexander
```

Текущий (рабочий) каталог можно определить с помощью команды pwd (print working directory).

Корневой каталог обозначается символом / (прямой слеш). Это самый верхний уровень в иерархии файловой системы.

2. Создать в начальном каталоге два подкаталога. Просмотреть содержимое рабочего каталога. Просмотреть содержимое родительского каталога, не переходя в него.

```
alexander@alexander-PC:~$ mkdir dir1 dir2
```

```
alexander@alexander-PC:~$ ls
'~'                                                    virtualbox_shared
 Android                                              'VirtualBox VMs'
 AndroidStudioProjects                                 Видео
 dir1                                                  Документы
 dir2                                                  Загрузки
 dotnet-install.sh                                     Изображения
 flutter                                               Музыка
 flutter_linux_3.29.0-stable.tar.xz                    Общедоступные
 Oracle_VM_VirtualBox_Extension_Pack-7.0.16.vbox-extpack  'Рабочий стол'
 snap                                                  Шаблоны
```

```
alexander@alexander-PC:~$ ls ..
alexander
```

3. Перейти в системный каталог. Просмотреть его содержимое. Просмотреть содержимое начального каталога. Вернуться в начальный каталог.

```
alexander@alexander-PC:~$ cd /etc
alexander@alexander-PC:/etc$ ls
adduser.conf            gtk-2.0              postgresql
alsa                    gtk-3.0              postgresql-common
alternatives            hdparm.conf          ppp
anacrontab              host.conf            printcap
apache2                 hostname             profile
apg.conf                hosts                profile.d
apm                     hosts.allow          protocols
apparmor                hosts.deny           pulse
apparmor.d              hp                   python3
apport                  ifplugd              python3.12
apt                     init                 rc0.d
avahi                   init.d               rc1.d
bash.bashrc             initramfs-tools      rc2.d
bash_completion         inputrc              rc3.d
bash_completion.d       insserv.conf.d       rc4.d
bindresvport.blacklist  ipp-usb              rc5.d
binfmt.d                iproute2             rc6.d
bluetooth               issue                rcS.d
brlapi.key              issue.net            resolv.conf
brltty                  kernel               rmt
brltty.conf             kerneloops.conf      rpc
ca-certificates         krb5.conf.d          rsyslog.conf
ca-certificates.conf    ldap                 rsyslog.d
chatscripts             ld.so.cache          rygel.conf
cloud                   ld.so.conf           sane.d
colord                  ld.so.conf.d         security
console-setup           legal                selinux
cracklib                libao.conf           sensors3.conf
credstore               libaudit.conf        sensors.d
credstore.encrypted     libblockdev          services
cron.d                  libibverbs.d         sgml
cron.daily              libnl-3              shadow
cron.hourly             libpaper.d           shadow-
cron.monthly            libreoffice          shells
crontab                 locale.alias         skel
cron.weekly             locale.conf          snmp
cron.yearly             locale.gen           speech-dispatcher
cups                    localtime            ssh
cupshelpers             logcheck             ssl
dbus-1                  login.defs           sssd
dconf                   logrotate.conf       subgid
debconf.conf            logrotate.d          subgid-
debian_version          lsb-release          subuid
```

```
alexander@alexander-PC:/etc$ ls ~
'~'                     dir1                 flutter
                        snap                 Видео        Изображения    'Рабочий
 стол'
 Android                dir2                 flutter_linux_3.29.0-stable.tar.xz
                        virtualbox_shared    Документы    Музыка         Шаблоны
 AndroidStudioProjects  dotnet-install.sh    Oracle_VM_VirtualBox_Extension_Pac
k-7.0.16.vbox-extpack  'VirtualBox VMs'      Загрузки     Общедоступные
```

```
alexander@alexander-PC:/etc$ cd
alexander@alexander-PC:~$
```

4. Удалить созданные ранее подкаталоги.

Получить информацию по командам ls и cd с помощью утилиты man. Изучить структуру man-документа.

```
alexander@alexander-PC:~$ rmdir dir1 dir2
alexander@alexander-PC:~$
```

```
LS(1)                          User Commands                          LS(1)

NAME
       ls - list directory contents

SYNOPSIS
       ls [OPTION]... [FILE]...

DESCRIPTION
       List  information  about the FILEs (the current directory by default).
       Sort entries alphabetically if none of -cftuvSUX nor --sort is  speci-
       fied.

       Mandatory  arguments  to  long options are mandatory for short options
       too.

       -a, --all
              do not ignore entries starting with .

       -A, --almost-all
              do not list implied . and ..
```

```
       The TIME_STYLE argument can be full-iso,  long-iso,  iso,  locale,  or
       +FORMAT.   FORMAT  is  interpreted like in date(1).  If FORMAT is FOR-
       MAT1<newline>FORMAT2, then FORMAT1 applies  to  non-recent  files  and
       FORMAT2  to recent files.  TIME_STYLE prefixed with 'posix-' takes ef-
       fect only outside the POSIX locale.  Also the  TIME_STYLE  environment
       variable sets the default style to use.

       The  WHEN  argument  defaults  to  'always'  and can also be 'auto' or
       'never'.

       Using color to distinguish file types is disabled both by default  and
       with --color=never.  With --color=auto, ls emits color codes only when
       standard output is connected to a terminal.  The LS_COLORS environment
       variable can change the settings.  Use the dircolors(1) command to set
       it.

   Exit status:
       0      if OK,

       1      if minor problems (e.g., cannot access subdirectory),

       2      if serious trouble (e.g., cannot access command-line argument).

AUTHOR
       Written by Richard M. Stallman and David MacKenzie.

REPORTING BUGS
       GNU coreutils online help: <https://www.gnu.org/software/coreutils/>
       Report any translation bugs to <https://translationproject.org/team/>

COPYRIGHT
       Copyright  ©  2023 Free Software Foundation, Inc.  License GPLv3+: GNU
       GPL version 3 or later <https://gnu.org/licenses/gpl.html>.
       This is free software: you are free to  change  and  redistribute  it.
       There is NO WARRANTY, to the extent permitted by law.

SEE ALSO
       dircolors(1)

       Full documentation <https://www.gnu.org/software/coreutils/ls>
       or available locally via: info '(coreutils) ls invocation'
```

```
alexander@alexander-PC:~$ man cd
Нет справочной страницы для cd
```

5.Получить краткую информацию по командам ls и cd с помощью команды whatis и apropos. В чем различие?

```
alexander@alexander-PC:~$ whatis ls
ls (1)               - list directory contents
alexander@alexander-PC:~$ whatis cd
cd: ничего подходящего не найдено.
```

```
alexander@alexander-PC:~$ apropos ls
proc_kallsyms (5)    - kernel exported symbols
_llseek (2)          - reposition read/write file offset
_Static_assert (3)   - fail compilation if assertion is false
aconnect (1)         - ALSA sequencer connection manager
add-shell (8)        - add shells to the list of valid login shells
afs_syscall (2)      - unimplemented system calls
alsa-info (8)        - command-line utility to gather information about the ...
alsabat (1)          - command-line sound tester for ALSA sound card driver
alsactl (1)          - advanced controls for ALSA soundcard driver
alsactl_init (7)     - alsa control management - initialization
alsaloop (1)         - command-line PCM loopback
alsamixer (1)        - soundcard mixer for ALSA soundcard driver, with ncurs...
```

```
alexander@alexander-PC:~$ apropos cd
proc_execdomains (5) - ABI personalities (obsolete)
apt-cdrom (8)        - APT CD-ROM management utility
cd-create-profile (1) - Color Manager Profile Creation Tool
cd-fix-profile (1)   - Color Manager Testing Tool
cd-it8 (1)           - Color Manager Testing Tool
dhcpcd (8)           - a DHCP client
dhcpcd-run-hooks (8) - DHCP client configuration script
dhcpcd.conf (5)      - dhcpcd configuration file
EVP_KEYEXCH-ECDH (7ssl) - ECDH Key Exchange algorithm support
EVP_SIGNATURE-ECDSA (7ssl) - The EVP_PKEY ECDSA signature implementation.
gcov-dump (1)        - offline gcda and gcno profile dump tool
gcov-dump-13 (1)     - offline gcda and gcno profile dump tool
```

6.То же, что и в п.5, только с помощью команды info.

```
Next: dir invocation,  Up: Directory listing

10.1 ‘ls’: List directory contents
==================================

The ‘ls’ program lists information about files (of any type, including
directories).  Options and file arguments can be intermixed arbitrarily,
as usual.  Later options override earlier options that are incompatible.

   For non-option command-line arguments that are directories, by
default ‘ls’ lists the contents of directories, not recursively, and
omitting files with names beginning with ‘.’.  For other non-option
arguments, by default ‘ls’ lists just the file name.  If no non-option
argument is specified, ‘ls’ operates on the current directory, acting as
if it had been invoked with a single argument of ‘.’.

   By default, the output is sorted alphabetically, according to the
locale settings in effect.(1)  If standard output is a terminal, the
output is in columns (sorted vertically) and control characters are
output as question marks; otherwise, the output is listed one per line
and control characters are output as-is.

   Because ‘ls’ is such a fundamental program, it has accumulated many
options over the years.  They are described in the subsections below;
within each section, options are listed alphabetically (ignoring case).
The division of options into the subsections is not absolute, since some
options affect more than one aspect of ‘ls’’s operation.

   Exit status:

     0 success
     1 minor problems  (e.g., failure to access a file or directory not
       specified as a command line argument.  This happens when listing a
       directory in which entries are actively being removed or renamed.)
     2 serious trouble (e.g., memory exhausted, invalid option, failure
       to access a file or directory specified as a command line argument
       or a directory loop)

   Also see *note Common options::.
```

```
alexander@alexander-PC:~$ info cd
info: Пункт меню «cd» отсутствует в ноде «(dir)Top»
```

Часть 2

1.Выполнить команды top, free, ps с различными опциями.

```
alexander@alexander-PC:~$ top

top - 21:12:04 up  2:47,  1 user,  load average: 0,27, 0,33, 0,18
Задачи: 359 total,   1 running, 358 sleeping,   0 stopped,   0 zombie
%Cpu(s):  2,7 us,  0,5 sy,  0,0 ni, 96,8 id,  0,0 wa,  0,0 hi,  0,0 si,  0,0 st
МиБ Mem :  15397,7 total,   9506,6 free,   3148,1 used,   2942,6 buff/cache
МиБ Swap:   4096,0 total,   4096,0 free,      0,0 used.  12249,6 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ КОМАНДА
   3903 alexand+  20   0 1433244 456044 176648 S  21,9   2,9   1:40.77 soffice.bin
   2913 alexand+  20   0 5562944 364128 165252 S  13,2   2,3   2:38.15 gnome-shell
   4154 alexand+  20   0  711752  66264  51300 S   1,7   0,4   0:11.42 gnome-terminal-
   8361 root      20   0       0      0      0 I   0,7   0,0   0:01.21 kworker/u49:1-flush-259:0
    282 root      20   0       0      0      0 I   0,3   0,0   0:00.44 kworker/9:1-mm_percpu_wq
   1691 root      20   0  139764  12320  11400 S   0,3   0,1   0:10.40 happd
   3069 alexand+  20   0  392092  12844   7480 S   0,3   0,1   0:04.19 ibus-daemon
   3248 alexand+  20   0  425332  31008  19552 S   0,3   0,2   0:02.56 ibus-extension-
   6200 alexand+  20   0 2747012 228344 114572 S   0,3   1,4   0:24.08 Isolated Web Co
   6616 root      20   0       0      0      0 I   0,3   0,0   0:01.46 kworker/u48:0-dm_vblank_control_workqueue
   9593 alexand+  20   0   17472   6128   3840 R   0,3   0,0   0:00.26 top
      1 root      20   0   24052  15276   9480 S   0,0   0,1   0:03.55 systemd
      2 root      20   0       0      0      0 S   0,0   0,0   0:00.01 kthreadd
      3 root      20   0       0      0      0 S   0,0   0,0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0,0   0,0   0:00.00 kworker/R-rcu_gp
      5 root       0 -20       0      0      0 I   0,0   0,0   0:00.00 kworker/R-sync_wq
      6 root       0 -20       0      0      0 I   0,0   0,0   0:00.00 kworker/R-kvfree_rcu_reclaim
      7 root       0 -20       0      0      0 I   0,0   0,0   0:00.00 kworker/R-slub_flushwq
      8 root       0 -20       0      0      0 I   0,0   0,0   0:00.00 kworker/R-netns
      9 root      20   0       0      0      0 I   0,0   0,0   0:01.62 kworker/0:0-events
     11 root       0 -20       0      0      0 I   0,0   0,0   0:00.00 kworker/0:0H-events_highpri
     13 root       0 -20       0      0      0 I   0,0   0,0   0:00.00 kworker/R-mm_percpu_wq
     14 root      20   0       0      0      0 I   0,0   0,0   0:00.00 rcu_tasks_kthread
     15 root      20   0       0      0      0 I   0,0   0,0   0:00.00 rcu_tasks_rude_kthread
     16 root      20   0       0      0      0 I   0,0   0,0   0:00.00 rcu_tasks_trace_kthread
     17 root      20   0       0      0      0 S   0,0   0,0   0:00.03 ksoftirqd/0
     18 root      20   0       0      0      0 I   0,0   0,0   0:04.36 rcu_preempt
     19 root      20   0       0      0      0 S   0,0   0,0   0:00.00 rcu_exp_par_gp_kthread_worker/0
     20 root      20   0       0      0      0 S   0,0   0,0   0:00.00 rcu_exp_gp_kthread_worker
     21 root      rt   0       0      0      0 S   0,0   0,0   0:00.07 migration/0
     22 root     -51   0       0      0      0 S   0,0   0,0   0:00.00 idle_inject/0
     23 root      20   0       0      0      0 S   0,0   0,0   0:00.00 cpuhp/0
     24 root      20   0       0      0      0 S   0,0   0,0   0:00.00 cpuhp/2
```

```
alexander@alexander-PC:~$ free
               всего        занят        своб      общая  буф/врем.   доступно
Память:     15767224      3197720      9760052      94904     3013988    12569504
Подкачка:    4194300            0      4194300
alexander@alexander-PC:~$ free -h
               всего        занят        своб      общая  буф/врем.   доступно
Память:         15Gi       3,0Gi       9,3Gi       92Mi       2,9Gi        11Gi
Подкачка:      4,0Gi          0B       4,0Gi
```

```
alexander@alexander-PC:~$ ps
    PID TTY          TIME CMD
   4163 pts/0    00:00:00 bash
   9737 pts/0    00:00:00 ps
alexander@alexander-PC:~$ ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.0  24052 15276 ?        Ss   18:24   0:03 /sbin/init splash
root           2  0.0  0.0      0     0 ?        S    18:24   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        S    18:24   0:00 [pool_workqueue_release]
root           4  0.0  0.0      0     0 ?        I<   18:24   0:00 [kworker/R-rcu_gp]
root           5  0.0  0.0      0     0 ?        I<   18:24   0:00 [kworker/R-sync_wq]
root           6  0.0  0.0      0     0 ?        I<   18:24   0:00 [kworker/R-kvfree_rcu_reclaim]
root           7  0.0  0.0      0     0 ?        I<   18:24   0:00 [kworker/R-slub_flushwq]
root           8  0.0  0.0      0     0 ?        I<   18:24   0:00 [kworker/R-netns]
root           9  0.0  0.0      0     0 ?        I    18:24   0:01 [kworker/0:0-mm_percpu_wq]
root          11  0.0  0.0      0     0 ?        I<   18:24   0:00 [kworker/0:0H-events_highpri]
root          13  0.0  0.0      0     0 ?        I<   18:24   0:00 [kworker/R-mm_percpu_wq]
root          14  0.0  0.0      0     0 ?        I    18:24   0:00 [rcu_tasks_kthread]
root          15  0.0  0.0      0     0 ?        I    18:24   0:00 [rcu_tasks_rude_kthread]
root          16  0.0  0.0      0     0 ?        I    18:24   0:00 [rcu_tasks_trace_kthread]
root          17  0.0  0.0      0     0 ?        S    18:24   0:00 [ksoftirqd/0]
root          18  0.0  0.0      0     0 ?        I    18:24   0:04 [rcu_preempt]
root          19  0.0  0.0      0     0 ?        S    18:24   0:00 [rcu_exp_par_gp_kthread_worker/0]
root          20  0.0  0.0      0     0 ?        S    18:24   0:00 [rcu_exp_gp_kthread_worker]
root          21  0.0  0.0      0     0 ?        S    18:24   0:00 [migration/0]
root          22  0.0  0.0      0     0 ?        S    18:24   0:00 [idle_inject/0]
root          23  0.0  0.0      0     0 ?        S    18:24   0:00 [cpuhp/0]
root          24  0.0  0.0      0     0 ?        S    18:24   0:00 [cpuhp/2]
root          25  0.0  0.0      0     0 ?        S    18:24   0:00 [idle_inject/2]
root          26  0.0  0.0      0     0 ?        S    18:24   0:00 [migration/2]
root          27  0.0  0.0      0     0 ?        S    18:24   0:00 [ksoftirqd/2]
root          28  0.0  0.0      0     0 ?        I    18:24   0:01 [kworker/2:0-mm_percpu_wq]
root          29  0.0  0.0      0     0 ?        I<   18:24   0:00 [kworker/2:0H-events_highpri]
root          30  0.0  0.0      0     0 ?        S    18:24   0:00 [cpuhp/4]
root          31  0.0  0.0      0     0 ?        S    18:24   0:00 [idle_inject/4]
root          32  0.0  0.0      0     0 ?        S    18:24   0:00 [migration/4]
root          33  0.0  0.0      0     0 ?        S    18:24   0:00 [ksoftirqd/4]
root          35  0.0  0.0      0     0 ?        I<   18:24   0:00 [kworker/4:0H-events_highpri]
root          36  0.0  0.0      0     0 ?        S    18:24   0:00 [cpuhp/6]
root          37  0.0  0.0      0     0 ?        S    18:24   0:00 [idle_inject/6]
root          38  0.0  0.0      0     0 ?        S    18:24   0:00 [migration/6]
root          39  0.0  0.0      0     0 ?        S    18:24   0:00 [ksoftirqd/6]
root          40  0.0  0.0      0     0 ?        I    18:24   0:00 [kworker/6:0-events]
```

2. Войти в свой домашний каталог. Для этого нужно сделать команду cd ~

Вы находитесь в своем рабочем каталоге. Здесь хранятся ваши пользовательские файлы и настройки программ, которые вы используете.

```
alexander@alexander-PC:~$ cd ~
alexander@alexander-PC:~$
```

3. Создать следующую структуру каталогов и файлов

1) в домашнем каталоге создать каталог inform

2) Перейти в каталог и inform создать в нем каталог lab1

3) Внутри каталога lab1 создать каталог catalog1, файл file1(например, используя

команду echo), каталог catalog2. Перейти в каталог catalog2.

4) Внутри каталога catalog2 создать файлы file3 и file4 , каталог catalog3

5) Внутри каталога catalog3 создать файл file5, жесткую ссылку на файл file1, жесткую ссылку на каталог catalog2.

6) Создать в каталоге lab1 символичесткую ссылку s_link на файл file5

```
alexander@alexander-PC:~$ mkdir inform
alexander@alexander-PC:~$ cd inform/
alexander@alexander-PC:~/inform$ mkdir lab1
alexander@alexander-PC:~/inform$
alexander@alexander-PC:~/inform$ cd lab1/
alexander@alexander-PC:~/inform/lab1$ mkdir catalog1 catalog2
alexander@alexander-PC:~/inform/lab1$ echo "Hello" > file1
alexander@alexander-PC:~/inform/lab1$ cd catalog2
alexander@alexander-PC:~/inform/lab1/catalog2$ mkdir catalog3
alexander@alexander-PC:~/inform/lab1/catalog2$ touch file3 file4
alexander@alexander-PC:~/inform/lab1/catalog2$ cd catalog3/
alexander@alexander-PC:~/inform/lab1/catalog2/catalog3$ touch file5
alexander@alexander-PC:~/inform/lab1/catalog2/catalog3$ ln ../../file1 hard_link_to_file1
alexander@alexander-PC:~/inform/lab1/catalog2/catalog3$ ln -s ../../catalog2 symlink_to_catalog2
alexander@alexander-PC:~/inform/lab1/catalog2/catalog3$ cd ~/inform/lab1
alexander@alexander-PC:~/inform/lab1$ ln -s catalog2/catalog3/file5 s_link
alexander@alexander-PC:~/inform/lab1$ ls -lR ~/inform/
/home/alexander/inform/:
итого 4
drwxrwxr-x 4 alexander alexander 4096 мар  4 22:56 lab1

/home/alexander/inform/lab1:
итого 12
drwxrwxr-x 2 alexander alexander 4096 мар  4 21:24 catalog1
drwxrwxr-x 3 alexander alexander 4096 мар  4 21:25 catalog2
-rw-rw-r-- 2 alexander alexander    6 мар  4 21:25 file1
lrwxrwxrwx 1 alexander alexander   23 мар  4 22:56 s_link -> catalog2/catalog3/file5

/home/alexander/inform/lab1/catalog1:
итого 0

/home/alexander/inform/lab1/catalog2:
итого 4
drwxrwxr-x 2 alexander alexander 4096 мар  4 22:55 catalog3
-rw-rw-r-- 1 alexander alexander    0 мар  4 21:25 file3
-rw-rw-r-- 1 alexander alexander    0 мар  4 21:25 file4

/home/alexander/inform/lab1/catalog2/catalog3:
итого 4
-rw-rw-r-- 1 alexander alexander  0 мар  4 21:25 file5
-rw-rw-r-- 2 alexander alexander  6 мар  4 21:25 hard_link_to_file1
lrwxrwxrwx 1 alexander alexander 14 мар  4 22:55 symlink_to_catalog2 -> ../../catalog2
```

```
alexander@alexander-PC:~/inform/lab1/catalog2/catalog3$ ln ../../catalog2 hard_link_to_catalog2
ln: ../../catalog2: не допускается создавать жёсткие ссылки на каталоги
```

Жёсткую ссылку создать невозможно, поэтому заменяем её на символическую.

Часть 3.

1. Создайте в домашнем каталоге следующую структуру подкаталогов (существующие каталоги не удаляйте!)

a. домашний каталог ----|

1. |-ВашаФамилия-|

a. |-1-|

b. | |-2

c. | |-3

d. |

e. |-4

```
alexander@alexander-PC:~/inform/lab1$ cd ~
alexander@alexander-PC:~$ mkdir -p Gornostaev/1/2 Gornostaev/1/3 Gornostaev/4
```

компанда mkdir -p создаёт все промежуточные каталоги

2. Скопируйте файл /etc/group в каталог 1 используя абсолютные имена копируемого файла и каталога назначения.

```
alexander@alexander-PC:~$ cp /etc/group ~/Gornostaev/1
```

3. Скопируйте файл /etc/group в каталог 2 используя абсолютное имя копируемого файла и относительное имя каталога назначения.

```
alexander@alexander-PC:~$ cp /etc/group ~/Gornostaev/1/2/
```

4. Скопируйте файл /etc/group в каталог 3 используя относительные имена копируемого файла и каталога назначения.

```
alexander@alexander-PC:~/Gornostaev$ cp /etc/group 1/3/
```

5. Скопируйте файл /etc/group в каталог 4 используя абсолютные имена копируемого файла и относительное имя каталога назначения с использованием специального символа ~.

```
alexander@alexander-PC:~/Gornostaev$ cp /etc/group ~/Gornostaev/4/
```

6. При помощи одной команды зайдите в каталог 3.

```
alexander@alexander-PC:~/Gornostaev$ cd ~/Gornostaev/1/3
alexander@alexander-PC:~/Gornostaev/1/3$
```

7. Удалите файл group из каталога 4 при помощи одной команды.

```
alexander@alexander-PC:~/Gornostaev/1/3$ rm ~/Gornostaev/4/group
```

8. Перейдите в свой домашний каталог. Удалите каталоги 1 и 4.

```
alexander@alexander-PC:~/Gornostaev/1/3$ cd
alexander@alexander-PC:~$ rm -r Gornostaev/1 Gornostaev/4
```

9. Выведите первые и последние 13 строк файла /etc/group.

```
alexander@alexander-PC:~$ head -n 13 /etc/group
root:x:0:
daemon:x:1:
bin:x:2:
sys:x:3:
adm:x:4:syslog,alexander
tty:x:5:
disk:x:6:
lp:x:7:
mail:x:8:
news:x:9:
uucp:x:10:
man:x:12:
proxy:x:13:
alexander@alexander-PC:~$ tail -n 13 /etc/group
polkitd:x:987:
rtkit:x:119:
colord:x:120:
gdm:x:121:
nm-openvpn:x:122:
lxd:x:123:
gamemode:x:986:
gnome-initial-setup:x:985:
alexander:x:1000:
snapd-range-524288-root:x:524288:
_daemon_:x:584792:
vboxusers:x:124:
postgres:x:125:
```

Часть 4

1. Создать группы:

- workers,
- teachers,
- students

```
alexander@alexander-PC:~$ sudo groupadd workers
[sudo] пароль для alexander:
alexander@alexander-PC:~$ sudo groupadd teachers
alexander@alexander-PC:~$ sudo groupadd students
```

Все команды, создающие/удаляющие пользователей и группы, требуют прав sudo.

2. Создать пользователей kit_[номер варианта]_ N, где N =1, 2, .., 5, uid учетной записи должен быть равен 1000+N.

Пользователей с N равным 1 и 2 добавить в группу workers.

Пользователей с N равным 3, 4 и 5 добавить в группу students.

```
alexander@alexander-PC:~$ sudo useradd -u 1001 -G workers kit_3_1
alexander@alexander-PC:~$ sudo useradd -u 1002 -G workers kit_3_2
alexander@alexander-PC:~$ sudo useradd -u 1003 -G students kit_3_3
alexander@alexander-PC:~$ sudo useradd -u 1004 -G students kit_3_4
alexander@alexander-PC:~$ sudo useradd -u 1005 -G students kit_3_5
```

```
alexander@alexander-PC:~$ echo "kit_3_1:Eofmzxk31" | sudo chpasswd
alexander@alexander-PC:~$ echo "kit_3_2:Eofmzxk31" | sudo chpasswd
alexander@alexander-PC:~$ echo "kit_3_3:Eofmzizx1" | sudo chpasswd
alexander@alexander-PC:~$ echo "kit_3_4:Eofmlzxx1" | sudo chpasswd
alexander@alexander-PC:~$ echo "kit_3_5:Eofmlzxx121" | sudo chpasswd
```

3. Создать пользователя teacher_[номер варианта]. В комментарии к учетной записи должны быть имя и фамилия студента. uid учетной записи должен быть равен 3000. Пользователя добавить в группу teachers.

```
alexander@alexander-PC:~$ sudo useradd -u 3000 -c "Alexander Gornostaev" -G teachers teacher_3
```

4. Создать директорию labs в корневом каталоге. В нем создать каталоги library и tests

```
alexander@alexander-PC:~$ mkdir -p /labs/tests
mkdir: невозможно создать каталог «/labs»: Отказано в доступе
alexander@alexander-PC:~$ sudo mkdir -p /labs/tests
alexander@alexander-PC:~$ sudo mkdir -p /labs/library
```

5. Создать файлы book_[фамилия студента]_N и поместить их в library

```
alexander@alexander-PC:~$ sudo touch /labs/library/book_gornostaev_1
alexander@alexander-PC:~$ sudo touch /labs/library/book_gornostaev_2
alexander@alexander-PC:~$ sudo touch /labs/library/book_gornostaev_3
alexander@alexander-PC:~$ sudo touch /labs/library/book_gornostaev_4
alexander@alexander-PC:~$ sudo touch /labs/library/book_gornostaev_5
```

6. Создать текстовый файл test_[имя студента], и поместить в tests. Файлы должны содержать скрипт на создание пользователя user[номер варианта] и задание ему пароля pass[номер варианта]. Сделайте эти файлы исполняемыми для пользователей группы students.

```
alexander@alexander-PC:~$ sudo bash -c 'cat > /labs/tests/test_gornostaev << EOF
#!/bin/bash
# Скрипт для создания пользователя user3 с паролем pass3
echo "Создание пользователя user3..."
sudo useradd user3 2>/dev/null
if [ $? -eq 0 ]; then
    echo "Пользователь user3 создан"
else
    echo "Пользователь user3 уже существует или ошибка создания"
fi
echo "user3:pass3" | sudo chpasswd
echo "Пароль pass3 установлен для пользователя user3"
echo "Скрипт выполнен успешно!"
EOF'
```

```
alexander@alexander-PC:~$ # Делаем файл исполняемым для группы students
sudo chmod 750 /labs/tests/test_ivan
# Меняем группу на students
sudo chown :students /labs/tests/test_ivan
```

7. В директории labs создать файл list, который должен содержать список файлов директории /etc.

```
alexander@alexander-PC:~$ sudo bash -c 'ls -la /etc > /labs/list'
```

8. Дать право на изменение файла только пользователю teacher_[номер варианта], а на чтение пользователям группы workers.

```
alexander@alexander-PC:~$ # 8. Настройка прав на файл list
# teacher_3 - полный доступ, группа workers - чтение, остальные - без доступа
sudo chown teacher_3:workers /labs/list
sudo chmod 740 /labs/list
```

9. Настроить права доступа к каталогу library и tests, таким образом, чтобы пользователи группы teachers могли изменять и создавать там файлы, а пользователи группы students имели доступ на чтение.

```
alexander@alexander-PC:~$ sudo chown :teachers /labs/library
alexander@alexander-PC:~$ sudo chmod 770 /labs/library
```

```
alexander@alexander-PC:~$ sudo bash -c 'chown :students /labs/library/*'
sudo bash -c 'chmod 640 /labs/library/*'
```

```
alexander@alexander-PC:~$ sudo chown :teachers /labs/tests
sudo chmod 775 /labs/tests
alexander@alexander-PC:~$ sudo chown :students /labs/tests/test_ivan
sudo chmod 750 /labs/tests/test_gornostaev
```

Проверка:

```
alexander@alexander-PC:~$ ls -ld /labs/library
drwxrwx--- 2 root teachers 4096 мар  4 23:57 /labs/library
```

```
alexander@alexander-PC:~$ sudo ls -l /labs/library/
итого 0
-rw-r----- 1 root students 0 мар  4 23:57 book_gornostaev_1
-rw-r----- 1 root students 0 мар  4 23:57 book_gornostaev_2
-rw-r----- 1 root students 0 мар  4 23:57 book_gornostaev_3
-rw-r----- 1 root students 0 мар  4 23:57 book_gornostaev_4
-rw-r----- 1 root students 0 мар  4 23:57 book_gornostaev_5
```

```
alexander@alexander-PC:~$ ls -ld /labs/tests
drwxrwxr-x 2 root teachers 4096 мар  5 00:09 /labs/tests
alexander@alexander-PC:~$ ls -l /labs/tests/
итого 8
-rwxr-x--- 1 root students 559 мар  5 00:09 test_gornostaev
```

Каталог library (770):

- 7 (владелец root): полный доступ (rwx)
- 7 (группа teachers): полный доступ (rwx) — могут изменять и создавать файлы
- 0 (остальные): нет доступа

Файлы в library (640):

- 6 (владелец root): чтение и запись
- 4 (группа students): только чтение — студенты имеют доступ на чтение
- 0 (остальные): нет доступа

Каталог tests (775):

- 7 (владелец root): полный доступ
- 7 (группа teachers): полный доступ — могут изменять и создавать файлы
- 5 (остальные, включая студентов): чтение и выполнение — студенты имеют доступ на чтение и могут заходить в каталог

Файл test_gornostaev (750):

- 7 (владелец root): полный доступ
- 5 (группа students): чтение и выполнение — студенты могут читать и выполнять скрипт
- 0 (остальные): нет доступа

Вывод

В ходе выполнения лабораторной работы мы научились работать в терминале Linux, освоили базовые команды такие как (ls, cd, mkdir, cp), также научились создавать группы пользователей в Linux.